

# Frontend Developer Portfolio

구조를 설계하고, 반복을 자동화하고, 실제 서비스까지 운영하는 프론트엔드 개발자

Architecture

Automation

Operation

Realtime UI

## Proof Objects

- Mason: Codex CLI 기반 AI harness
- Kakao Map: 격자 분할 완전 탐색
- Snapshot: 주문/결제/재고/알림 운영 구조
- F-Lab: 기업회원 채용 제휴 도메인
- 냉코 & FilmMate: App Store 출시 앱
- Bullet Samurai: Phaser/Electron 데스크톱 게임

# 저는 이런 개발자입니다.

프론트엔드에 머무르지 않고, 제품이 운영되는 전체 흐름을 직접 설계해왔습니다.

사수나 체계적인 교육 없이 공식 문서와 기술 커뮤니티를 기반으로 커리어를 시작했습니다. 필요한 기술이라면 프론트엔드에만 머무르지 않고 백엔드, 인프라, 배포와 운영까지 직접 부딪히며 배워왔습니다.

입사 2년 차부터 리드 프론트엔드 역할을 맡아 대규모 업무 시스템의 구조를 잡고, 반복되는 개발을 줄이는 공통 컴포넌트와 설정 기반 화면 구조를 정리했습니다.

혼자 성장하는 과정의 어려움을 알기 때문에, 후배 개발자들이 더 빠르게 방향을 잡고 성장할 수 있도록 적극적으로 고민하고 소통하려고 합니다.

## WORKING STYLE

### 구조화

데이터 구조, 상태 흐름, 권한 체계까지 함께 설계

### 자동화

반복되는 화면 설계와 구현 과정을 CLI 파이프라인으로 연결

### 운영 감각

서비스 출시 이후 주문, 알림, 어드민, 장애 대응 흐름까지 고려

# 수치로 남은 구현 경험

실제 제품, 업무 시스템, 자동화 도구에서 검증된 결과를 중심으로 정리했습니다.

5건

공기업·대기업·정부기관 대상 대규모 시스템  
구축

80%+

공기업 K사 안전보건 플랫폼 FE 코드 직접  
개발

3,000만원

React-Quill 커스터마이징으로 유료 솔루션  
대체

3,000건+

Snapshot 개인 서비스 누적 주문 처리

35곳 / 164건

F-Lab 기업회원·이력서 초기 수요 검증

CLI

설계서·코드·캡처·히스토리 통합 자동화

## 문제를 구조로 바꾼 6개의 사례



# Mason AI Harness CLI

Codex CLI가 작업 규칙과 요구사항 문맥을 읽고, Mason이 이를 코드·캡처·화면설계서로 고정합니다.

## PROJECT INTRO

Codex CLI가 작업 규칙과 요구사항 문맥을 읽고, Mason이 이를 코드·캡처·화면설계서로 고정합니다.

## SECTION FLOW

- 01 문맥 해석
- 02 규칙 고정
- 03 검증 가능한 산출물

# Codex CLI를 감싸 업무 화면 생성 흐름으로 연결했습니다

Codex가 작업 규칙과 요구사항 문맥을 읽고, Mason이 이를 검증 가능한 산출물로 고정합니다.



포인트: Codex CLI의 해석 능력을 Mason의 규칙/검증 계층으로 감싸, AI 결과를 바로 개발 산출물로 이어지게 만든 harness입니다.

**프로덕션 화면과 PPTX 설계서가 함께 산출됩니다**

같은 spec에서 리스트/모달 화면을 구현하고, 캡처 기반 화면설계서까지 바로 이어서 만듭니다.

[illegible]

**DETAIL MODAL**

상세 입력 품과 검증 흐름을 포함한 모달 화면

**Production Screen**

첨부파일

첨가

**PPTX Design Doc**

요약 내용

첨부파일

첨가

필수 (Mandatory)  
선택 (Optional)

# AI가 만든 초안을 개발 가능한 산출물로 고정했습니다

Codex CLI가 자유로운 요구사항을 해석하고, Mason은 스펙 검증·디자인시스템 매칭·코드/문서 생성을 담당합니다.

|                              |                                                      |                           |                                                                      |
|------------------------------|------------------------------------------------------|---------------------------|----------------------------------------------------------------------|
| <b>Codex Harness</b>         | 작업 규칙과 문맥을 Codex CLI에 전달해 요구사항 해석을 표준화               | <b>Spec Normalization</b> | 자연어 요구사항을 화면/흐름 명세로 고정해 구현 기준을 명확화                                   |
| <b>Design System Binding</b> | Storybook과 사내 컴포넌트 사용을 전제로 생성 결과를 제한                 | <b>Production Code</b>    | Next.js page, setting, columns, forms, modal view까지 실제 반영 가능한 구조로 산출 |
| <b>QA / History</b>          | Playwright 캡처, Codex QA, 작업 히스토리를 남겨 재현 가능한 리뷰 흐름 확보 |                           |                                                                      |



# Kakao Map Grid Search

Kakao Map API의 조회 결과 제한 때문에 누락될 수 있는 주변 시설 데이터를 4분할 재귀 탐색으로 끝까지 수집한 지도 알고리즘 사례입니다.

## PROJECT INTRO

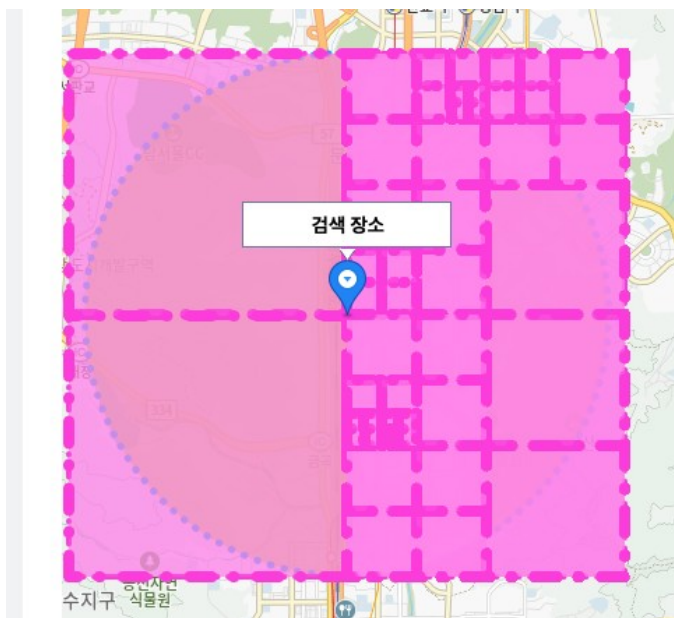
Kakao Map API의 조회 결과 제한 때문에 누락될 수 있는 주변 시설 데이터를 4분할 재귀 탐색으로 끝까지 수집한 지도 알고리즘 사례입니다.

## SECTION FLOW

- 01 문제와 제약 조건
- 02 45건 포화 판정
- 03 4분할 재귀 탐색

# 단일 반경 조회 제한을 격자 분할 완전 탐색으로 풀었습니다

보험 가입 사업장 주변 수용체 데이터를 누락 없이 확보하기 위한 프론트엔드 알고리즘 설계입니다.



## 알고리즘 구조

1. 중심 좌표에서 radius=3000m로 category/keyword API 조회
2. 3page까지 조회했는데 pageable\_count가 45면 결과가 잘렸다고 판단
3. 중심 기준 3km Bounding Box를 만들고 rect를 4등분
4. 각 rect도 45건 포화 조건이 사라질 때까지 재귀 분할 후 수집

핵심 성과: API가 한 번에 반환하지 못하는 밀집 구역을 4분할 재귀 탐색으로 끝까지 수집

# Snapshot

스마트폰 대여 서비스를 고객 주문 화면, 결제, 재고, 락커, 관리자 대시보드, 운영 알림까지 연결한 웹 기반 렌탈 운영 시스템입니다.

## PROJECT INTRO

스마트폰 대여 서비스를 고객 주문 화면, 결제, 재고, 락커, 관리자 대시보드, 운영 알림까지 연결한 웹 기반 렌탈 운영 시스템입니다.

## SECTION FLOW

- 01 서비스/인프라 구조
- 02 주문 도메인 분리
- 03 운영 상태 흐름

[서비스 보러가기](#)

# SNS 운영을 웹 기반 렌탈 운영 시스템으로 전환했습니다

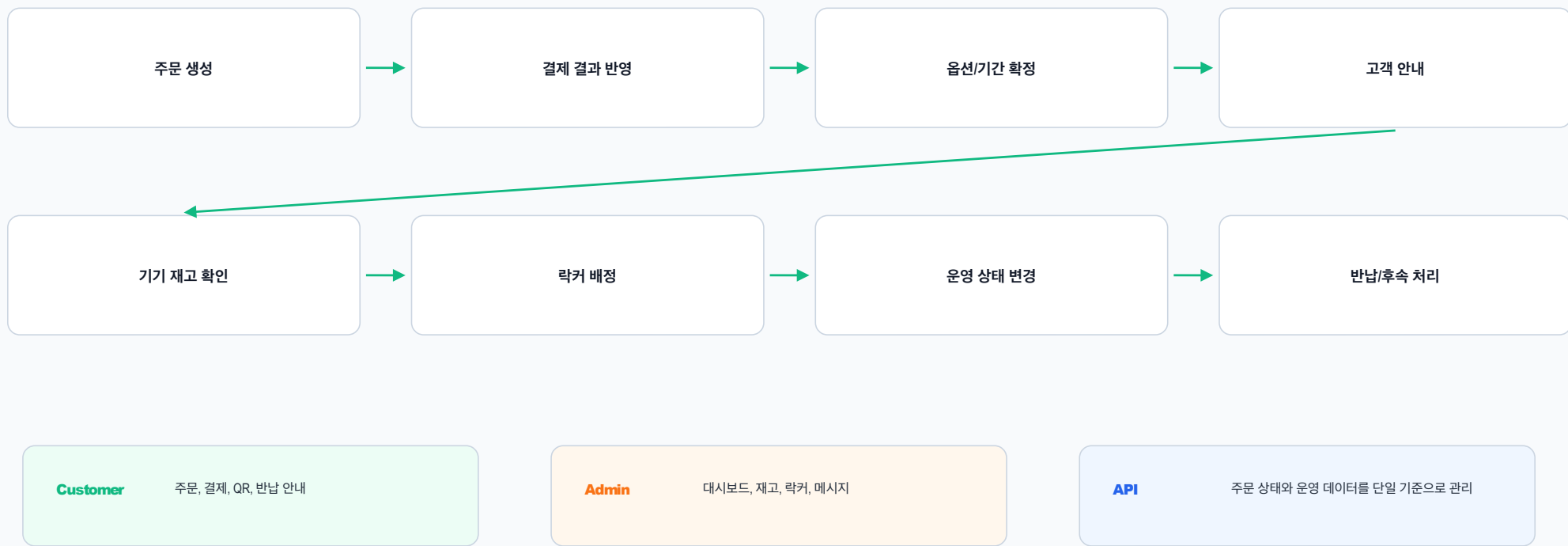
고객 주문 화면, NestJS API, 관리자 화면, AWS 인프라가 분리되어 운영 흐름을 만듭니다.



|                           |                               |                               |
|---------------------------|-------------------------------|-------------------------------|
| <b>주문</b> 예약, 옵션, 상태 전이   | <b>결제</b> 결제 결과와 주문 상태 연결     | <b>재고</b> 기기 모델, 수량, 대여 가능 여부 |
| <b>락커</b> 픽업/반납 위치와 재고 배정 | <b>알림</b> 운영 Slack, 고객 안내 메시지 | <b>운영</b> 대시보드, 일괄 처리, 검색/필터  |

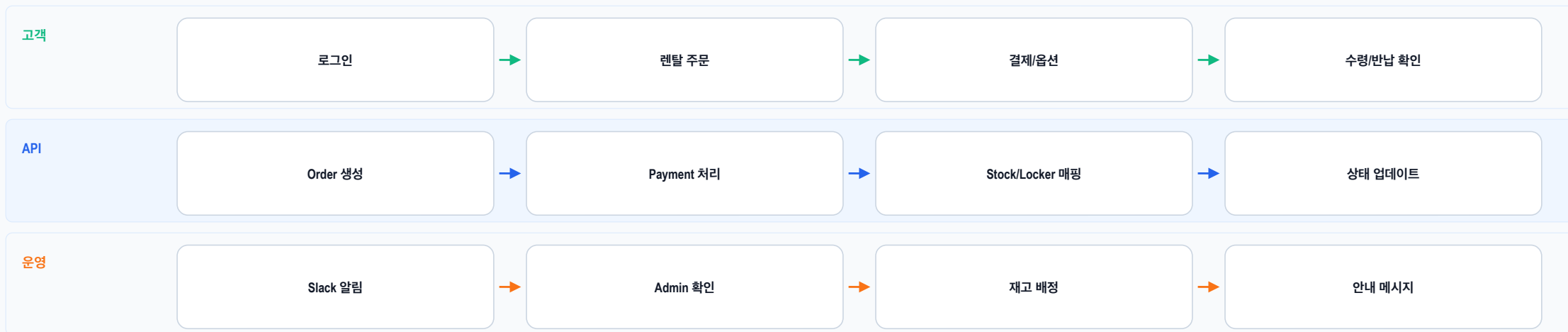
## 주문 하나가 결제, 재고, 락커, 알림, 운영 화면으로 확장됩니다

렌탈 서비스는 주문 생성 이후의 운영 처리가 더 중요하기 때문에 도메인을 작게 나누고 상태 흐름을 연결했습니다.



# 주문 이후의 운영 처리를 자동화 가능한 상태 전이로 정리했습니다

고객 화면에서 끝나는 서비스가 아니라, 운영자가 처리할 수 있는 백오피스 흐름까지 설계했습니다.



운영 포인트: 주문 상태 변경이 고객 안내, 재고 배정, 락커 처리, 관리자 대시보드에 동시에 반영되도록 흐름을 단일화했습니다.

# F-Lab 기업회원

기존 F-Lab 회원 인프라 위에 기업회원 신청, 관리자 검수, 멘티 이력서 공개, 채용 제안 수락/거절까지 확장한 B2B 채용 제휴 기능입니다.

## PROJECT INTRO

기존 F-Lab 회원 인프라 위에 기업회원 신청, 관리자 검수, 멘티 이력서 공개, 채용 제안 수락/거절까지 확장한 B2B 채용 제휴 기능입니다.

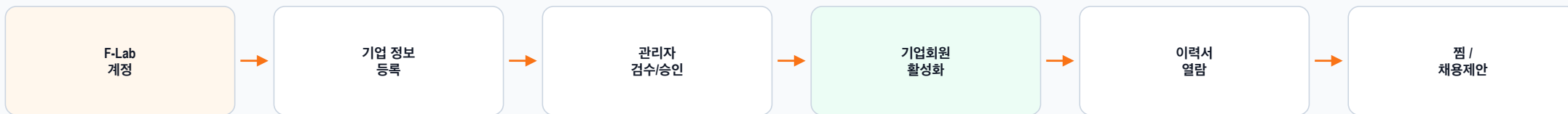
## SECTION FLOW

- 01 기업회원 가입/검수
- 02 멘티 LMS 연동
- 03 이력서/제안 도메인

[서비스 보러가기](#)

# 기존 회원 인프라에 기업회원 채용 도메인을 확장했습니다

기업 계정, 백오피스 검수, 멘티 LMS 이력서, 채용 제안 흐름을 하나의 business 도메인으로 연결했습니다.



## 기업회원

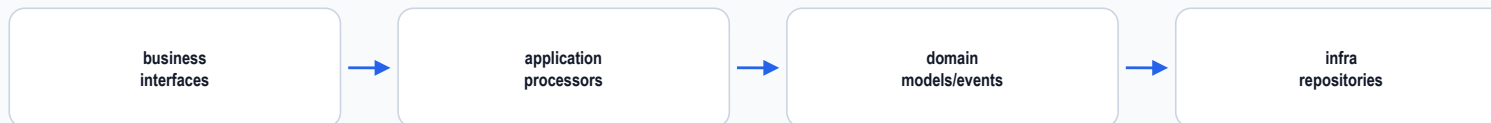
에프랩 계정으로 기업 정보와 사업자 정보를 등록합니다. 승인 이후에는 멘티 이력서 목록을 탐색하고, 관심 멘티를 찜하거나 채용 제안 품을 보낼 수 있습니다.

## 에프랩 관리자

백오피스 business 화면에서 기업회원 신청 정보를 확인하고 승인/거절 상태를 변경합니다. 상태 변경은 기업회원 권한과 서비스 접근 가능 여부로 이어집니다.

## 멘티 LMS

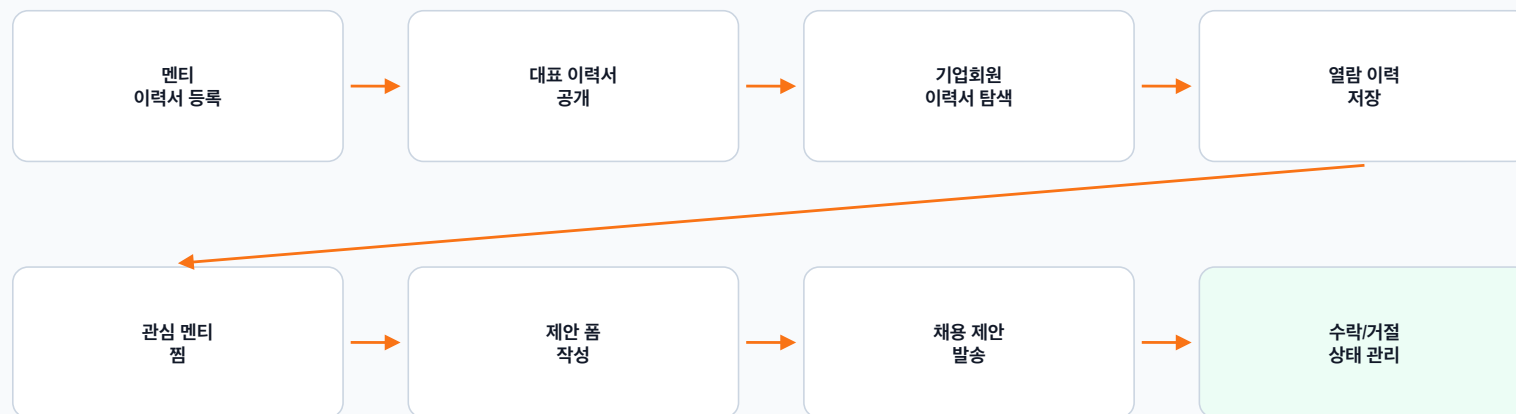
멘티는 LMS employment 페이지에서 이력서를 등록/수정하고 공개 여부를 관리합니다. 기업에서 온 제안을 열람한 뒤 수락하거나 거절할 수 있습니다.





# 기업회원과 멘티가 서로 이어지는 채용 제휴 워크플로우

화면은 business 페이지, 백오피스, LMS로 나뉘지만 데이터는 business-api의 이력서/제안 도메인으로 모입니다.



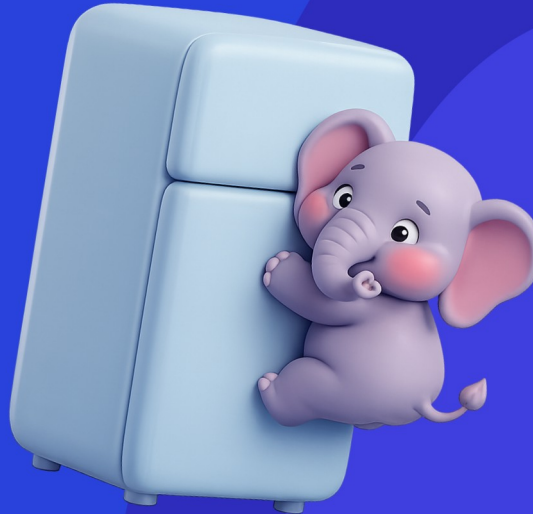
## 구현 도메인

|                 |                    |
|-----------------|--------------------|
| <b>Business</b> | 기업 정보와 검수 상태       |
| <b>Profile</b>  | 멘티 이력서와 공개 여부      |
| <b>Offer</b>    | 제안 품, 제안 발송, 수락/거절 |
| <b>History</b>  | 이력서 열람/수정 이력       |

핵심: 신규 화면 추가에 그치지 않고 기업 승인, 멘티 이력서 공개, 열람 이력, 제안 상태까지 실제 채용 제휴 도메인으로 모델링했습니다.



냉장고 속 재료들,  
냉코가 챙겨줄게요



# 식재료 입력 과정을 여러 경로로 줄였습니다



사용자는 영수증 스캔, 바코드 인식, 직접 입력 중 상황에 맞는 방식을 선택하고, 결과 화면에서 카테고리 및 보관 위치, 소비기한을 바로 정리합니다.

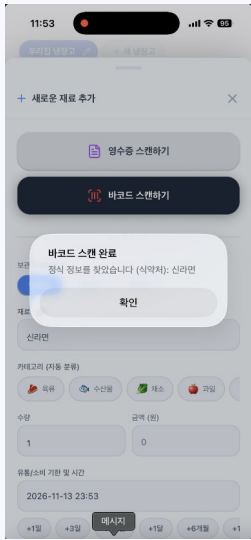
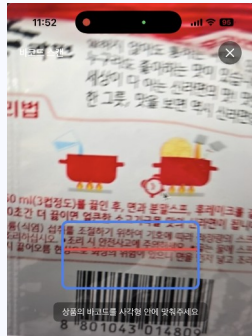
## 영수증 스캔

영수증을 촬영해 글자를 읽고, 추출된 여러 상품을 한 번에 확인한 뒤 냉장고에 넣을 항목을 선택합니다.



## 바코드 스캔

상품 바코드를 프레임 안에 맞추면 정식 상품명 입력 폼에 반영되고, 사용자는 수량, 금액, 소비기한만 보정하면 됩니다.



# AI 셰프가 보유 재료를 레시피와 영상 추천으로 연결합니다

재료와 취향을 선택하고, 추천 결과를 확인한 뒤, 유튜브 영상까지 이어지는 흐름입니다.



## 1. 재료와 취향 선택

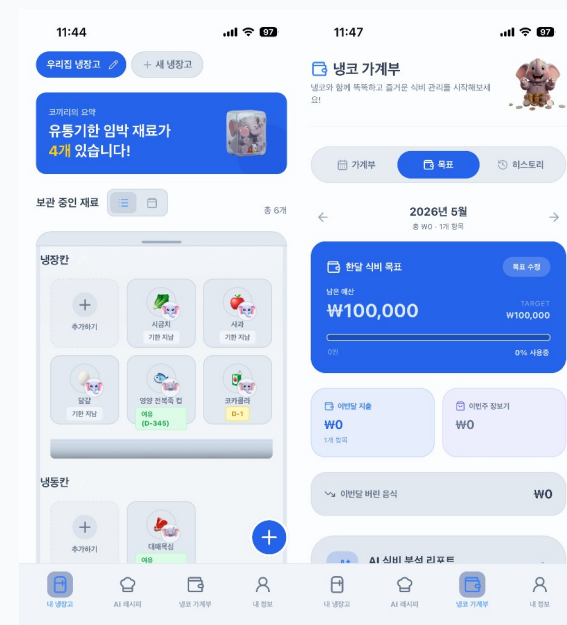
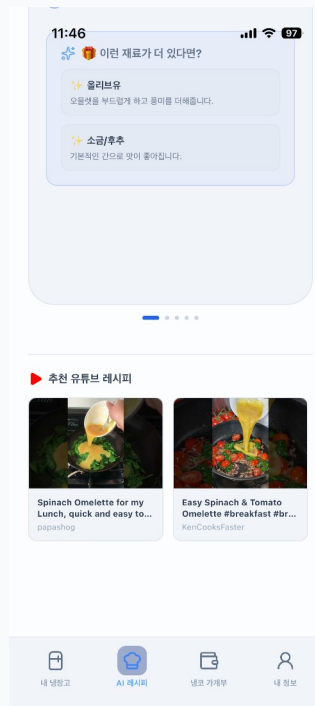
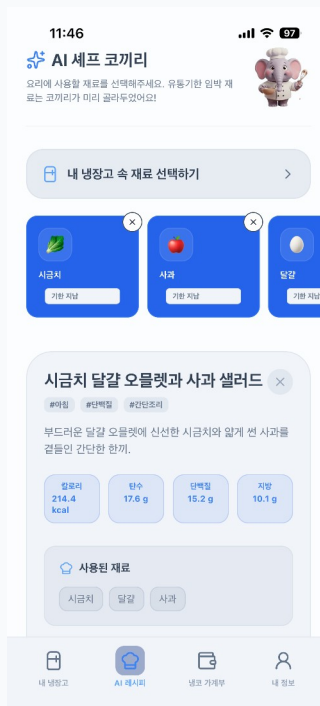
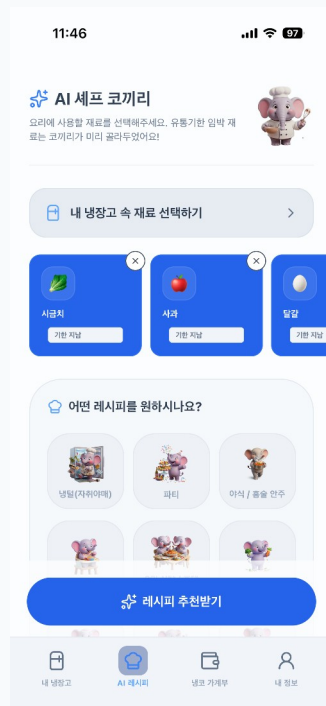
냉장고 속 재료 중 요리에 사용할 항목을 선택하고, 유통기한이 임박한 재료를 우선 노출해 관리 행동이 추천으로 이어지게 했습니다.

## 2. 추천 결과 확인

선택한 재료와 레시피 타입을 기반으로 음식명, 설명, 칼로리와 영양 정보를 카드 형태로 보여줍니다.

## 3. 영상 연결

레시피 결과에서 바로 참고할 수 있는 유튜브 영상을 연결해 사용자가 앱 안에서 다음 행동을 이어갈 수 있게 했습니다.



# FILM MATE

페어링 & 모니터



# 역할 선택부터 QR 페어링, 촬영 요청까지 이어지는 흐름

카메라 기기와 모니터 기기가 서로 다른 역할을 맡고, QR 기반 페어링 이후 WebRTC 화면 공유로 연결됩니다.



# WebRTC로 한 기기의 카메라 화면을 다른 기기에서 공유합니다

Firebase signaling과 개인 VPS STUN/TURN 중계 흐름을 연결해 원격 촬영 경험을 구현했습니다.

## 역할 기반 UX

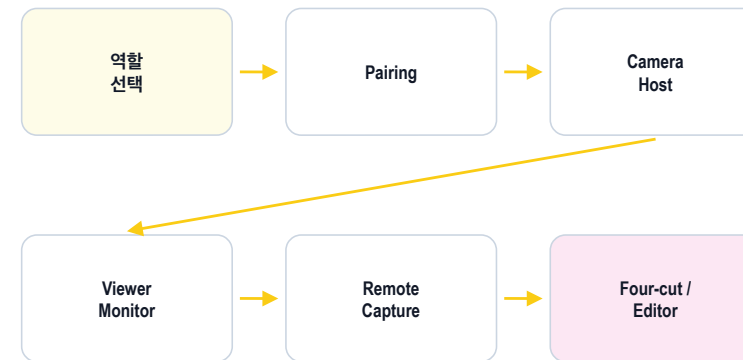
CAMERA는 카메라 스트림을 송출하고 MODEL은 QR 페어링 후 화면을 모니터링합니다.  
한 사람이 혼자 찍거나 둘이 함께 찍는 사용성을 모두 고려했습니다.

## 연결 안정성

Firebase에 offer/answer/ICE candidate를 저장해 시그널링을 처리하고, NAT 환경에서는  
개인 VPS의 STUN/TURN 중계로 연결 가능성을 높였습니다.

## 촬영 경험 확장

실시간 화면 공유 이후 촬영 요청, 포토부스 프레임, 사진 편집/LUT 흐름까지 이어지도록  
단일 모바일 경험으로 구성했습니다.



기능: 실시간 카메라 공유 · 원격 촬영 · 인생네컷 · 필름 LUT 편집

# Firestore는 연결 정보를 교환하고, STUN/TURN은 네트워크 장벽을 넘습니다

WebRTC 미디어 스트림은 P2P를 우선하고, NAT 환경에서는 VPS TURN relay로 안정성을 확보합니다.





# BULLET SAMURAI

*Press the space bar to select.*



**GAME START**

**HOW TO PLAY**

**RANKING**

# Phaser로 만든 탄막 패링 게임을 Electron 데스크톱 앱으로 포팅했습니다

적의 총알을 검 타이밍으로 받아내는 반응형 전투 구조와 STOVE Store 배포 경험을 함께 정리했습니다.

Phaser

Electron

Ranking

STOVE

## 게임 루프

GameStart에서 시작해 QuickDrawBullet에서 전투하고, 결과는 Ranking과 GameOver로 마무리하는 흐름으로 구성했습니다.

## 데스크톱 포팅

client 빌드 결과를 Electron으로 감싸고 electron-builder로 패키징해 데스크톱 실행 가능한 형태로 정리했습니다.

## 배포

웹 데모가 아니라 STOVE Store에 등록된 실제 배포형 게임으로 정리해 출시 경험을 남겼습니다.



Enemies fire guns at regular intervals.

### TUTORIAL 001

탄막을 읽고 검 타이밍으로 받아치는 패링 액션

Phaser · Electron · Firebase Firestore · electron-builder

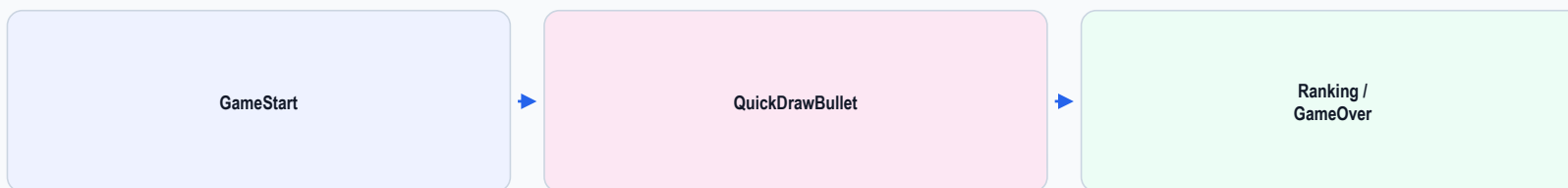


You can slash bullets by pressing the space bar.

### TUTORIAL 002

# GameStart, QuickDrawBullet, Ranking, GameOver로 이어지는 흐름

게임의 핵심 전투 구조와 결과 흐름을 한 줄로 정리했습니다.



## 전투 구조

화면 중앙에서 날아오는 탄막을 읽고 검 타이밍에 맞춰 받아치는 패링 액션입니다. 탄막 생성, 이동 속도, 피격 판정, 점수/라이프 갱신이 실시간으로 이어집니다.

## 데스크톱 포팅 / 배포

Phaser로 구현한 게임 클라이언트를 Electron으로 감싸 데스크톱 앱 형태로 배포했고, STOVE Store 출시 경험까지 연결했습니다.

## 랭킹 / 피드백

Firebase Firestore에 랭킹을 저장·조회하고, 게임 종료 후 재도전이 자연스럽게 이어지도록 결과 흐름을 분리했습니다.

# 문제를 구조로 바꾸고, 구조를 제품과 운영으로 연결합니다.

대규모 업무 시스템, AI 자동화 CLI, 실서비스 운영, 모바일 앱 출시까지 프론트엔드가 영향을 줄 수 있는 범위를 계속 넓혀왔습니다.

GitHub: [github.com/Joji6666](https://github.com/Joji6666)

Portfolio: JIN'S WORLD

Email: [huseng8810@gmail.com](mailto:huseng8810@gmail.com)